

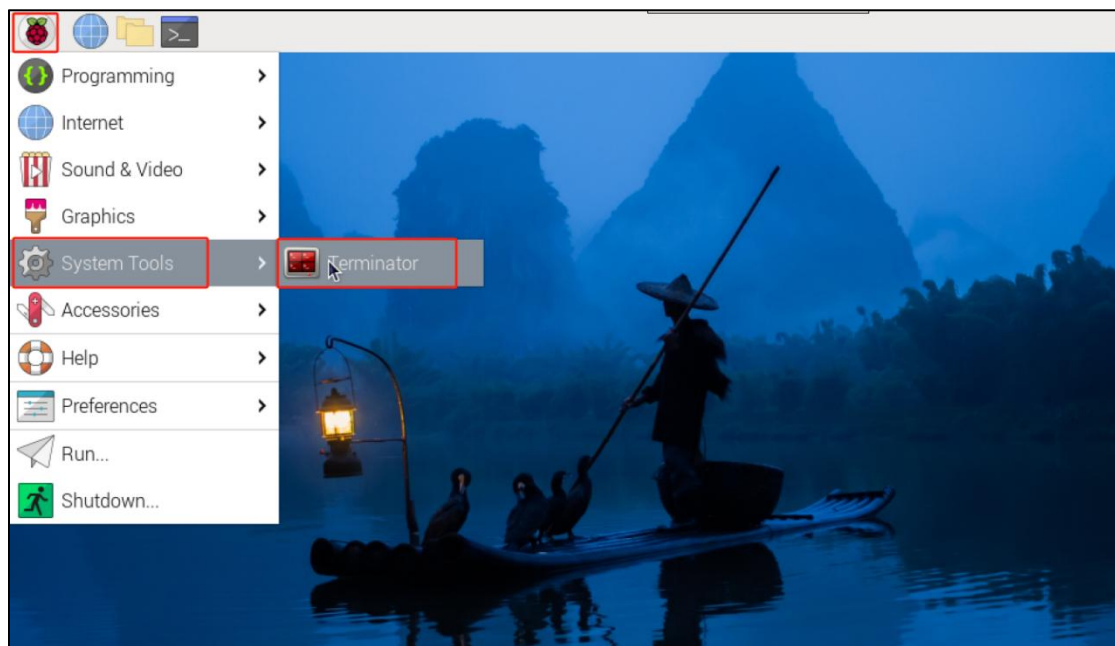
Lesson 7 ROS2 Node

1. Node Introduction

During communication, regardless of the method used, the construction of communication entities relies on nodes. In ROS2, each node typically corresponds to a single function module. For example, a lidar driver node may be responsible for publishing lidar messages, while a camera driver node may be responsible for publishing image messages. A complete robotic system may consist of many collaborating nodes. A single executable file in ROS 2 (either a C++ program or a Python program) can contain one or more nodes.

2. Create Node

1) Click  in the upper left corner, and then select “**System Tools and Terminator**” in sequence.



2) Enter the command “**cd**
hiwonder_ws/src/hello_world_demo/hello_world_demo” and press Enter
to switch to the path of the “**hello_world_demo**” package.

```
ubuntu@raspberrypi:~$ cd hiwonder_ws/src/hello_world_demo/hello_world_demo
ubuntu@raspberrypi:~/hiwonder_ws/src/hello_world_demo/hello_world_demo$
```

3) Enter command “**package**” to edit program. Copy the below program.
You can press “**i**” to edit the program. After modification, press “**Esc**” and enter
“**:wq**” to save and exit the program.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/hello_world_demo/hello_world_demo$ vim hello_world.py
```

```
import rclpy

from rclpy.node import Node

import time

class HelloWorldNode(Node):

    def __init__(self):

        # Call the constructor of the based class Node to set the node name.

        super().__init__('hello_world_demo')

    def run(self):

        # Execute a loop when the ROS2 system is running normally.

        while rclpy.ok():

            # Print “Hello World” to the node’s log.

            self.get_logger().info('Hello World')

            # Sleep for 0.5s to control the loop time.
```

```
time.sleep(0.5)

def main(args=None):

    # Initialize ROS2 Python interface

    rclpy.init(args=args)

    # Create an instance of HelloWorldNode

    node = HelloWorldNode()

    try:

        # Run the main loop of the node.

        node.run()

    except KeyboardInterrupt:

        pass

    finally:

        # Destroy the node object

        node.destroy_node()

        # Shut down ROS2 Python interface

        rclpy.shutdown()

if __name__ == '__main__':

    # Execute the main function

    main()
```

```
if __name__ == '__main__':  
    # 执行主函数  
    main()  
:wq
```

4) Enter the command “**chmod +x hello_world.py**” and press Enter to grant the executable permission to the saved hello_world.py file.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/hello_world_demo/hello_world_demo$ chmod +x  
hello_world.py  
ubuntu@raspberrypi:~/hiwonder_ws/src/hello_world_demo/hello_world_demo$
```

3. Compilation and Execution

1) After granting the executable permission, enter the command “**cd ~/hiwonder_ws/**” to switch to the directory of the workspace.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/hello_world_demo$ cd ~/hiwonder_ws/  
ubuntu@raspberrypi:~/hiwonder_ws$
```

2) Enter the command “**colcon build**” and press Enter to compile the package within the workspace.

```
ubuntu@raspberrypi:~/hiwonder_ws$ colcon build
```

3) Enter the command “**colcon build**” and press Enter to make the environment variables take effect.

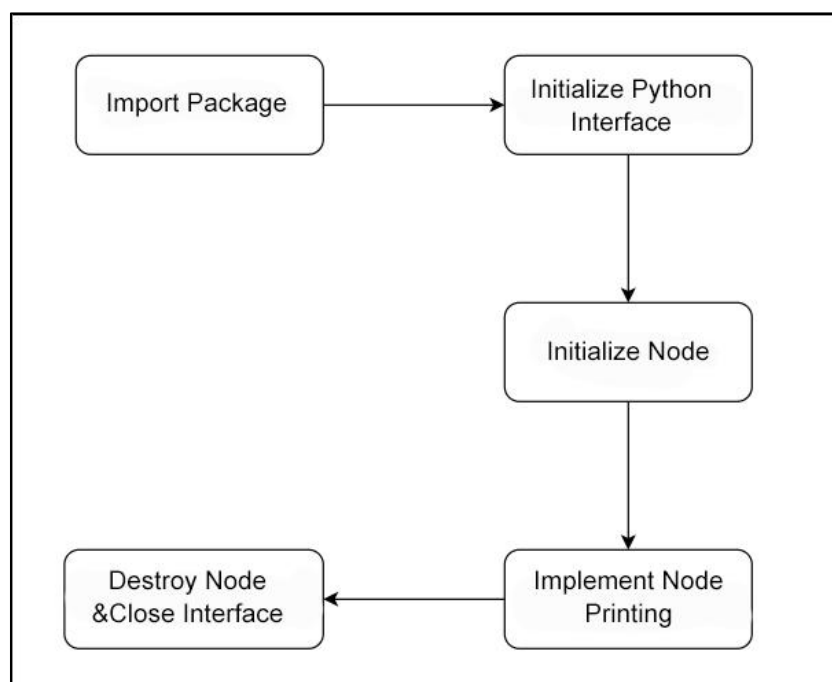
```
ubuntu@raspberrypi:~/hiwonder_ws$ source ./install/setup.bash
```

4) Enter command “**ros2 run hello_world_demo hello_world**” and press Enter to start hello_world node.

```
ubuntu@raspberrypi: ~  
ubuntu@raspberrypi: ~ 80x24  
ubuntu@raspberrypi:~$ ros2 run hello_world_demo hello_world  
[INFO] [1706176110.897927832] [hello_world_demo]: Hello World  
[INFO] [1706176111.399216463] [hello_world_demo]: Hello World  
[INFO] [1706176111.900952555] [hello_world_demo]: Hello World  
[INFO] [1706176112.402359428] [hello_world_demo]: Hello World  
[INFO] [1706176112.903731597] [hello_world_demo]: Hello World  
[INFO] [1706176113.405050415] [hello_world_demo]: Hello World  
[INFO] [1706176113.906528363] [hello_world_demo]: Hello World  
[INFO] [1706176114.407898590] [hello_world_demo]: Hello World  
[INFO] [1706176114.909127374] [hello_world_demo]: Hello World  
[INFO] [1706176115.410359343] [hello_world_demo]: Hello World  
[INFO] [1706176115.911586961] [hello_world_demo]: Hello World  
[INFO] [1706176116.412792673] [hello_world_demo]: Hello World  
[INFO] [1706176116.913994922] [hello_world_demo]: Hello World
```

4. Program Analysis

According to the realization result, the logic progress for the program is shown as pictured:



Create a node named HelloWorldNode, and print “Hello World” to the node’s log every 0.5 seconds in the main loop of the node. During program execution, the ROS2 Python interface is initialized first, then a node instance is created and its main loop is run. When the program is interrupted, the node object is

destroyed, and the ROS2 Python interface is shut down.

Main function

```
def main(args=None):
    # 初始化 ROS2 Python 接口
    rclpy.init(args=args)
    # 创建 HelloWorldNode 实例
    node = HelloWorldNode()

    try:
        # 运行节点的主循环
        node.run()
    except KeyboardInterrupt:
        pass
    finally:
        # 销毁节点对象
        node.destroy_node()
        # 关闭 ROS2 Python 接口
        rclpy.shutdown()
```

First, invoke the `rclpy.init()` function to initialize ROS2 Python interface. Then instantiate the `HelloWorldNode` file. Finally, execute the `node.run()` function.

◆ HelloWorldNode Class

```
class HelloWorldNode(Node):
    def __init__(self):
        # 调用基类 Node 的构造函数，设置节点名称
        super().__init__('hello_world_demo')

    def run(self):
        # 在 ROS2 系统正常运行的情况下执行循环
        while rclpy.ok():
            # 打印 "Hello World" 到节点的日志
            self.get_logger().info('Hello World')
            # 休眠 0.5 秒，控制循环时间
            time.sleep(0.5)
```

First, create a node named `HelloWorldNode`, then print “Hello World” to the node’s log at intervals of 0.5 seconds in the node’s main loop.